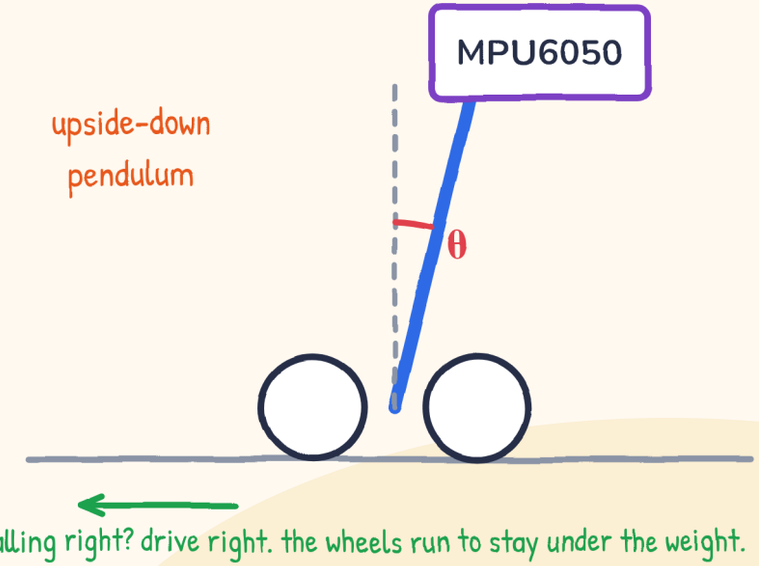
Robotics · Motors & Motion Control

Balancing Robot

Segways stay upright by falling forward and catching themselves, a hundred times a second.

120 minutes | Pairs | Code Lab + PID Lab + real kit



Did you know?

A two-wheeled balancing robot is an inverted pendulum, the same problem as balancing a broom on your palm. It is unstable by design: left alone it always falls.

Let's think

Balance a broom on your hand

Try it, right now

Stand a ruler or a broom upright on your palm. To keep it up you must move your hand **towards the direction it is falling**.

Notice two things: you never stop moving, and you react to *lean*, not to position.

Now try it with your eyes shut

Impossible. Without measurement there is no control. The robot's eyes are an MPU6050.

Guess first

- Is a long broom easier or harder than a short one?
- What happens if you react too strongly?
- What happens if you react a moment too late?
- Why can nobody balance a pencil?

The answers are the whole of control theory, in your hand.

LEARN

Learning aims

Today I will learn to...

1. Read **tilt** from an MPU6050 and explain why one sensor is not enough
2. Combine accelerometer and gyro with a **complementary filter**
3. Balance an **inverted pendulum in simulation** and tune it
4. Build and tune a real **two-wheel balancing robot**

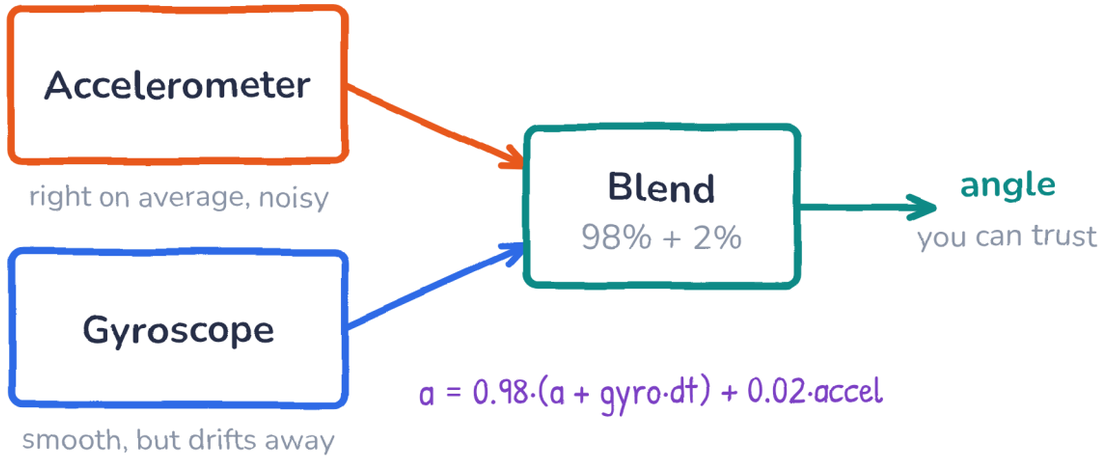
Keywords

inverted pendulum · IMU · accelerometer · gyroscope · drift · sensor fusion · complementary filter · PID · setpoint · overshoot

Objective 1 · The sensor

Two sensors, both wrong

Each fixes the other's fault



Accelerometer

Measures which way gravity pulls, so it always knows true vertical. But every motor vibration shakes it, so the reading jumps about.

Gyroscope

Measures how fast you are rotating. Add it up over time and you get an angle: beautifully smooth, but it **drifts** a degree every few seconds.

Fusion

Trust the gyro for the short term and the accelerometer for the long term. One line of code does it.

Objective 2 · The filter

One line that makes it work

```
angle = 0.98 * (angle + gyro * dt)
      + 0.02 * accelAngle;
```

Read it out loud

"Mostly, carry on from where I was and add how far I just rotated. But nudge 2% of the way towards what gravity says, every time."

Why 0.98?

It sets how long the filter remembers the gyro. At 100 Hz, 0.98 means roughly half a second of trust before gravity wins. Too high and drift creeps in; too low and vibration gets through.

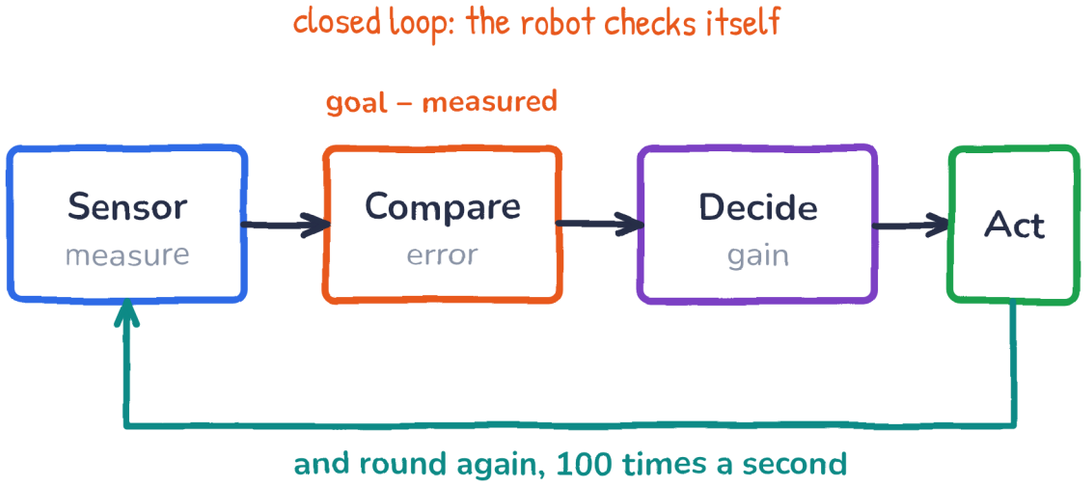
dt matters

Gyro gives degrees *per second*. Multiply by the real loop time. If your loop time wanders, your angle wanders with it, so fix the loop at a steady 10 ms.

Objective 3 · Control

Measure, compare, correct

The loop that never stops



Error is the only input

error = **0°** (upright) – measured angle. The controller never thinks about "upright"; it only ever reduces a number towards zero.

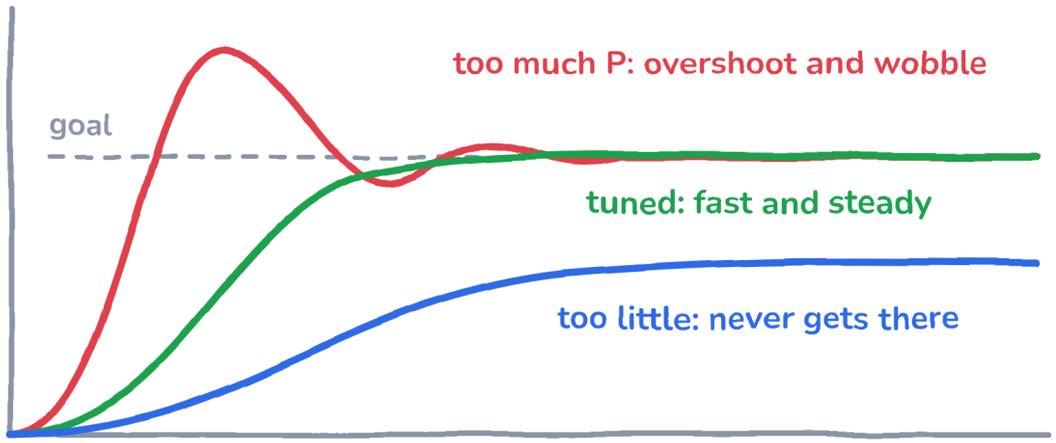
P, I and D

P pushes in proportion to the lean now. **I** clears up a small permanent lean. **D** brakes, by reacting to how fast the lean is changing.

Objective 3 · Tuning

What a bad gain looks like

Read the shape, not the feeling



change one gain at a time, and write down what happened

Too much P

Over-corrects, shoots past upright, over-corrects the other way. The robot shakes itself to the floor.

Too little P

Leans and keeps leaning. It never catches up with its own fall.

The method

Raise P until it oscillates. Then add D until the oscillation stops. Only then add a little I. One change at a time, written down.

In the real world

Inverted pendulums you have met

You, standing up

Your ankles make tiny corrections constantly. Stand still and feel them. Your inner ear is the IMU.

A rocket on launch

Thrust is below the mass, so it is an inverted pendulum too. The engine gimbals to keep it pointing up.

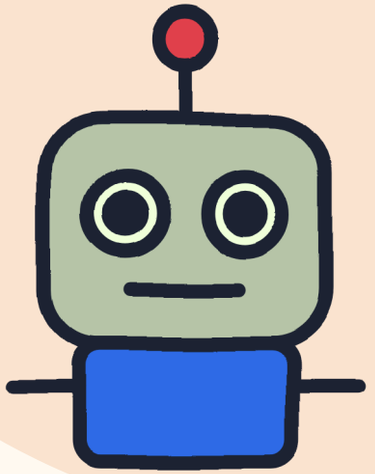
A Segway

Lean forward and the wheels drive forward to get back under you. You are steering by deliberately falling.

Part 2 · Make it

Simulate, then build

Tune it where nothing can break, then move the numbers onto a real robot.



80 minutes | Pairs | PID Lab then kit

Before you build

What you need



Kit, per pair

- MPU6050 IMU
- 2WD chassis, rebuilt **tall**: battery at the top
- L298N motor driver
- Arduino Uno + USB cable
- 18650 cell + holder



Build it tall on purpose

A short robot falls too fast for an Uno to catch. Put the heavy battery **as high as you can**.

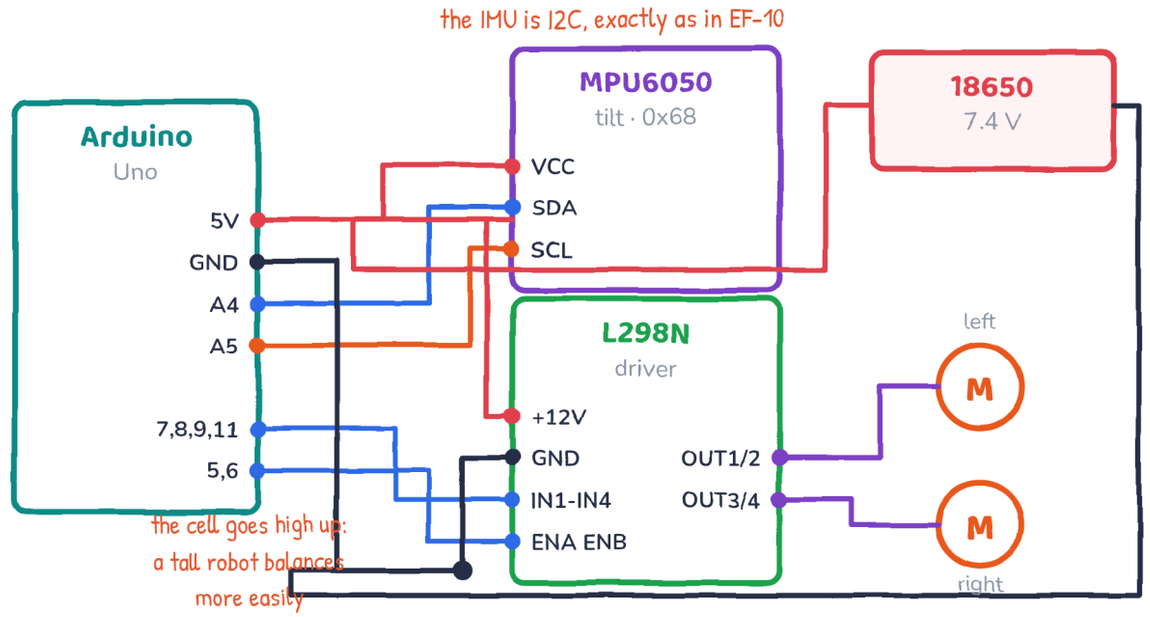
This is the broom-versus-pencil question from the start of the lesson, and now it is your design decision.

Keep fingers away from the wheels. Test over a soft surface.

Wiring

Sensor and motors on one board

IMU on I2C, motors through the driver



Connections

	Arduino Uno	Module	Carries
●	`5V`	`MPU VCC`	power for the IMU
●	`A4`	`MPU SDA`	tilt data
●	`A5`	`MPU SCL`	the I2C clock
●	`7, 8, 9, 11`	`IN1–IN4`	motor directions
●	`5, 6`	`ENA, ENB`	motor speed (PWM)
●	`GND`	`all GND`	one shared reference

The 18650 feeds +12V on the driver. Its minus joins the same ground as everything else.

Build step 1 of 4

Read the angle

Do this

1. Wire the MPU6050 to A4 and A5 (it is I2C: see EF-10).
2. Print the raw accelerometer angle. Tilt it. Tap the desk.
3. Print the gyro-only angle. Leave it still for a minute.
4. Now print the fused angle and compare all three.

Watch the drift

Leave the gyro-only version running while you do step 2. Come back and see how far it has wandered.

```
float angle = 0;
unsigned long last = 0;

float readAngle() {
  float dt = (millis() - last) / 1000.0;
  last = millis();

  float aAng = atan2(ay, az) * 57.2958;
  float gRate = gx / 131.0;     // deg/sec

  angle = 0.98 * (angle + gRate * dt)
        + 0.02 * aAng;
  return angle;
}
```

Build step 2 of 4

Tune it in the PID Lab

Do this

1. Open **Simulators → PID Lab** and choose the pendulum.
2. Set I and D to zero. Raise P until it wobbles forever.
3. Halve P. Now raise D until the wobble dies away.
4. Nudge the pendulum. Does it recover in under a second?
5. Write your three numbers on the board.

Why simulate first

In the simulator a bad gain costs you nothing. On the desk it costs a snapped motor bracket and ten minutes.

Compare across the class

Every pair will land on slightly different numbers. That is correct: the gains belong to the machine, not to the theory.

Build step 3 of 4

Close the loop for real

Do this

1. Hold the robot upright and note the angle it reads. That is your true zero.
2. Start with the PID Lab numbers, divided by two.
3. Hold it, power on, and let go of it slowly.
4. Change **one** gain. Try again. Log what happened.

Safety cutout

Past 35 degrees the robot cannot recover. Cut the motors instead of letting it thrash on the floor.

```
float Kp = 22, Ki = 0.4, Kd = 1.4;
float integral = 0, prev = 0;

void loop() {
  float a = readAngle();
  float err = 0 - a;            // 0 = upright

  integral += err * 0.01;
  integral = constrain(integral, -50, 50);
  float deriv = (err - prev) / 0.01;
  prev = err;

  int out = Kp*err + Ki*integral + Kd*deriv;
  out = constrain(out, -255, 255);

  if (abs(a) > 35) out = 0;     // given up
  drive(out, out);
  delay(10);                    // steady 100 Hz
}
```

Build step 4 of 4

Keep a tuning log

Try	Kp	Ki	Kd	What it did
1	22	0	0	Shook fast and fell in under a second
2	11	0	0	Leaned slowly and fell, no shaking
3	16	0	0.8	Held for 3 seconds, then drifted off
4	16	0.4	1.4	Held, recovered from a nudge
5	…	…	…	Your turn

Why the log matters

Without it you will change two things at once, it will work, and you will never know which one did it.

Test like an engineer

Prove it is really controlling

Try this	What should happen	What it teaches
Nudge it gently	It leans, then recovers	That is closed loop working
Put a book on top	It needs more Kp to hold	Gains belong to the machine, not the maths
Tilt the floor slightly	It creeps downhill and holds	I is correcting a permanent error
Set Kd to zero	The wobble returns	D is the brake
Drop the loop to 20 Hz	It fails completely	Control is about speed as much as gains

Battery voltage counts

Gains that worked on a full cell will be too weak on a flat one. Retune, or note the voltage next to every log entry.

LEVEL UP

Finished early?

Go further

1 

Add **remote steering** over Bluetooth by shifting the setpoint

Ask it to lean 2 degrees and it drives forward.

2 

Plot the angle live with the **Serial Plotter**

Now you can see overshoot instead of guessing at it.

3 

Add **encoders** so it holds its position, not just its angle

Two loops, one inside the other.

4 

Measure **recovery time** after a standard nudge

A real number to compare your tunings with.

Check yourself

Can I really do it? Prove it.

1 Read **tilt** from an MPU6050

Proof: I showed accelerometer noise and gyro drift separately

2 Fuse the two with a **complementary filter**

Proof: my fused angle was steady and did not drift

3 Balance an inverted pendulum **in simulation**

Proof: it recovered from a nudge in the PID Lab

4 Build and tune a **real balancing robot**

Proof: it stood up, and my tuning log explains why

Rate yourself: red = not yet · amber = nearly · green = yes, and I can show someone

Show what you know

What did we learn?

Five things to keep

- A balancing robot is an inverted pendulum: unstable by nature.
- Accelerometers are noisy; gyros drift. Fuse them.
- Error is the only thing a controller ever sees.
- P pushes, D brakes, I clears up what is left.
- Tune one gain at a time, and write it down.

Exit ticket

Your robot shakes rapidly and falls. Which gain would you change first, up or down, and why?

At home

Watch a lift arrive at a floor. It slows before it stops. That is D, working in a building.